D. A-B-C Sort

time limit per test 2 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1200 | Tags: constructive algorithms, implementation, sortings | https://codeforces.com/problemset/problem/1674/D

You are given three arrays a, b and c. Initially, array a consists of n elements, arrays b and c are empty.

You are performing the following algorithm that consists of two steps:

- Step 1: while a is not empty, you take *the last element* from a and move it *in the middle* of array b. If b currently has odd length, you can choose: place the element from a to the left or to the right of the middle element of b. As a result, a becomes empty and b consists of n elements.
- Step 2: while b is not empty, you take *the middle element* from b and move it *to the end* of array c. If b currently has even length, you can choose which of two middle elements to take. As a result, b becomes empty and c now consists of n elements.

Refer to the Note section for examples.

Can you make array c sorted in non-decreasing order?

Input

The first line contains a single integer t ($1 \le t \le 2 \cdot 10^4$) — the number of test cases. Next t cases follow.

The first line of each test case contains the single integer n ($1 \le n \le 2 \cdot 10^5$) — the length of array a.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^6$) — the array a itself.

It's guaranteed that the sum of n doesn't exceed $2 \cdot 10^5$.

Output

For each test, print YES (case-insensitive), if you can make array c sorted in non-decreasing order. Otherwise, print NO (case-insensitive).

Example

Input
3 4 3 1 5 3 3 3 2 1 1 7331

Output
YES NO YES

Note

In the first test case, we can do the following for a = [3, 1, 5, 3]:

Step 1: *a*[3, 1, 5, 3]□[3, 1, 5]□[3, 1]□[3]□[]*b*[][underline3][3, underline5][3, underline1, 5][3, underline3, 1, 5]

Step 2: *b*[3, 3, underline1, 5]□[3, underline3, 5]□[underline3, 5]□[underline5]□[]*c*[][1][1, 3][1, 3, 3][1, 3, 3, 5] As a result, array c = [1, 3, 3, 5] and it's sorted.